

SOURCE CODE

Student Performance Analytics

Note: This is a complete sample implementation for the project. It can be adapted to your actual dataset.

1. Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

print("Libraries imported successfully")
```

2. Create / Load Dataset

```
# Load the CSV dataset
# Keep student_performance.csv in the same folder as this program.

df = pd.read_csv("student_performance.csv")

print("Dataset loaded successfully")
print(df.head())
print(df.shape)
```

3. Check Dataset Information

```
print("Column names:")
print(df.columns)

print("\nDataset information:")
print(df.info())

print("\nMissing values:")
print(df.isnull().sum())

print("\nStatistical summary:")
print(df.describe())
```

4. Clean the Data

```
# Remove duplicate records
df = df.drop_duplicates()

# Fill missing numerical values with the column average
numeric_columns = [
    "Age", "Study_Hours", "Attendance",
    "Assignment_Score", "Internal_Marks",
    "Previous_Marks", "Final_Marks"
]

for column in numeric_columns:
    df[column] = df[column].fillna(df[column].mean())

# Fill missing categorical values
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])

print("Data cleaning completed")
print(df.isnull().sum())
```

5. Calculate Performance Percentage

```
# Calculate an overall score from academic components
df["Overall_Score"] = (
    df["Assignment_Score"] * 0.20 +
    df["Internal_Marks"] * 0.30 +
```

```

        df["Final_Marks"] * 0.50
    )

df["Percentage"] = df["Overall_Score"]

print(df[["Student_ID", "Percentage"]].head())

```

6. Create Performance Categories

```

def performance_category(score):
    if score >= 80:
        return "Excellent"
    elif score >= 60:
        return "Good"
    elif score >= 40:
        return "Average"
    else:
        return "Needs Improvement"

df["Performance"] = df["Percentage"].apply(performance_category)

print(df["Performance"].value_counts())

```

7. Basic Performance Analysis

```

print("Average Attendance:",
      round(df["Attendance"].mean(), 2), "%")

print("Average Final Marks:",
      round(df["Final_Marks"].mean(), 2))

print("Average Study Hours:",
      round(df["Study_Hours"].mean(), 2))

print("\nPerformance categories:")
print(df["Performance"].value_counts())

```

8. Top Performing Students

```

top_students = df.sort_values(
    by="Percentage", ascending=False
).head(10)

print("Top 10 Students:")
print(top_students[
    ["Student_ID", "Percentage", "Performance"]
])

```

9. Attendance vs Performance Graph

```

plt.figure(figsize=(8, 5))

plt.scatter(
    df["Attendance"],
    df["Final_Marks"],
    alpha=0.7
)

plt.xlabel("Attendance (%)")
plt.ylabel("Final Marks")
plt.title("Attendance vs Final Marks")
plt.grid(True)
plt.show()

```

10. Performance Category Graph

```

performance_counts = df["Performance"].value_counts()

plt.figure(figsize=(8, 5))
performance_counts.plot(kind="bar")

plt.xlabel("Performance Category")
plt.ylabel("Number of Students")

```

```
plt.title("Student Performance Distribution")
plt.xticks(rotation=0)
plt.show()
```

11. Average Marks by Performance

```
average_marks = df.groupby(
    "Performance"
)["Final_Marks"].mean()

print(average_marks)

plt.figure(figsize=(8, 5))
average_marks.plot(kind="bar")

plt.xlabel("Performance Category")
plt.ylabel("Average Final Marks")
plt.title("Average Final Marks by Performance")
plt.xticks(rotation=0)
plt.show()
```

12. Prepare Data for Machine Learning

```
# Features used to predict performance
features = [
    "Age",
    "Study_Hours",
    "Attendance",
    "Assignment_Score",
    "Internal_Marks",
    "Previous_Marks"
]

X = df[features]
y = df["Performance"]

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())
```

13. Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

14. Train Decision Tree Model

```
model = DecisionTreeClassifier(
    random_state=42,
    max_depth=5
)

model.fit(X_train, y_train)

print("Model training completed")
```

15. Test the Model

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:",
```

```
        round(accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

16. Predict a New Student's Performance

```
new_student = pd.DataFrame({
    "Age": [20],
    "Study_Hours": [3.5],
    "Attendance": [85],
    "Assignment_Score": [78],
    "Internal_Marks": [72],
    "Previous_Marks": [75]
})

prediction = model.predict(new_student)

print("Predicted Performance:",
      prediction[0])
```

17. Save the Processed Dataset

```
df.to_csv(
    "processed_student_performance.csv",
    index=False
)

print("Processed dataset saved successfully")
```

18. Conclusion

This source code implements the main functions of the Student Performance Analytics project. It loads and cleans student data, calculates performance, creates performance categories, generates visualizations, and applies a Decision Tree classification model. The system can be extended with a Streamlit dashboard, database connectivity, additional machine-learning models, and automated reports.